

The Substrate Near-Capacity Boundary as Standalone Architectural Treatment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one boundary case in which the pattern's cost-scaling, tool-agnosticism, retraceability, and governance commitments are jointly stressed: the case in which the substrate approaches its storage, query, or operational capacity limits while the deployment must continue to operate.

Abstract

The CKS pattern's commitments to linear-cost scaling, tool-agnosticism, path retraceability, and human governance each carry their own architectural content under normal operating conditions. At the boundary where the substrate approaches its storage, query, or operational capacity limits, all four are stressed jointly, and several auto-resolution patterns drawn from non-governed data infrastructure — auto-shedding, auto-pruning, capacity-driven LLM mediation, vendor-managed expansion — become available as immediate-relief mechanisms that silently move authority from humans to runtime or vendor. This note formalizes the architectural treatment: capacity decisions become governance moments handled by human-authored orchestration rules; horizontal scaling, vendor migration, and partitioning/archival are admissible responses; performance variance under capacity pressure is recorded as bounded non-determinism; historical-state preservation is non-negotiable. The note distinguishes performance variance from architecture violation, enumerates seven anti-pattern responses, and provides an operational test for whether a deployment handles the boundary in a CKS-coherent way.

1. Why the substrate near-capacity boundary needs to be formalized as standalone

Substrate-level capacity limits are operationally common at the scale most CKS deployments grow into. Persistent structured state with full provenance metadata grows monotonically; query throughput grows with cell count multiplied by participant activity; operational demand grows with the number of human and LLM participants reading and writing the substrate. Any of these axes eventually approaches whatever capacity ceiling the host environment carries — storage cap, indexing query rate, read/write throughput. The CKS pattern does not commit to any specific ceiling; what it commits to is a

particular handling of the moment at which a ceiling is approached.

That handling is not obvious. The non-CKS default — the pattern an AI-native substrate might inherit from data-platform conventions outside the AI domain — is to auto-resolve: handlers that auto-shed load, auto-prune oldest content, auto-compress historical detail, auto-expand into vendor-managed elasticity, or auto-fail over into vendor-specific scaling tiers. None is admissible inside CKS without further constraint, because each silently moves authority from humans to vendor or runtime at the moment of pressure, and several violate retraceability or source-of-truth commitments directly.

Standalone formalization matters because the boundary stresses four foundational commitments at once, and it is at exactly such moments that a deployment is most likely to silently lose architectural integrity. Source paper §6.2's cost-curve commitment is what is most directly under load; §3.3's authority-vs-labor framing of governance is what determines whether the response preserves or violates the architecture; §11.3's substrate-as-source-of-truth commitment is what auto-pruning would directly contravene. A6.07 is the seventh of approximately fifteen Phase A6 boundary case notes. Prior notes formalize boundary cases at adjacent surfaces — vendor unavailability, LLM consultation timeout, composition partner failure, and other operational discontinuities. This note formalizes the case in which the substrate itself, rather than a system around it, is the locus of pressure.

2. The boundary case scenario

The boundary case obtains when substrate operations begin showing capacity-related stress without yet failing. Indicators may be present individually or in combination: read or write operations completing with growing latency relative to baseline; query throughput approaching the indexing infrastructure's ceiling; storage allocation approaching the configured cap, or projected to reach it within the planning horizon; vendor-specific scaling cliffs visible as step-changes in cost or behavior at the next tier; monitoring metrics showing capacity pressure even when no operation has yet failed.

What makes the boundary non-obvious is that the deployment must continue operating while the capacity issue is addressed. Stopping is rarely an option; the substrate is the coordination artifact cells and human participants depend on. The architectural question is what kinds of response are admissible while operation continues, and what kinds compromise the architecture even though they appear to relieve the pressure. The most operationally tempting responses — those that relieve pressure quickly and without operator action — are the ones the architecture rules out.

3. Which architectural commitments are stressed

The boundary case stresses four foundational commitments jointly, and naming each precisely is what determines what counts as an admissible response.

A1.06 — linear-cost scaling — under capacity pressure. The cost-curve commitment (source paper §6.1, §6.2) is that adding coordination knowledge incurs linear infrastructure cost — database-like addition rather than parameter growth, re-embedding, or coordination overhead. Near a capacity ceiling, per-operation cost may show super-linear behavior as the host's indexing structures saturate. The commitment is a property of the design, not of the host's instantaneous behavior; the boundary case

is where the gap between design and instantaneous behavior is most visible. The commitment is preserved across the boundary by the response choices that follow, not by the fiction that no variance is occurring.

A1.05 — tool-agnosticism — for capacity migration. The commitment (§7.1) is that any environment satisfying three minimal requirements — persistent structured state, human read/write access, LLM access to substrate content — can host the pattern. When a host approaches its ceiling, migration to a higher-capacity host is one admissible response. The migration must preserve substrate content and provenance fully; tool-agnosticism is what makes it a substrate-level operation rather than a vendor-locked transition.

A1.07 — path retraceability — under data-volume pressure. Provenance metadata grows with the substrate; the per-content metadata fields are themselves substrate content, and their accumulation contributes to the pressure that triggers the boundary. Retraceability does not shrink under capacity pressure. Archival is the legitimate mechanism for managing storage, and archival rules must preserve full provenance and full addressability of archived content.

A1.01 — human-governance — for capacity decisions. Decisions about how to address capacity — expand, migrate, partition, archive, or accept the variance — are architectural decisions. Under §3.3's framing of governance as authority architecture rather than review workflow, these are governance moments and require rule authoring per the architecture's rule-authoring commitment. A vendor- or runtime-driven auto-resolution moves the locus of authority from the humans who hold inspect, modify, and override rights to the vendor or runtime, regardless of whether any individual operation visibly fails.

4. The architectural treatment

The treatment is the joint application of several mechanisms, each sitting inside a commitment the source paper already defends. None is novel to the boundary; what is novel is naming their joint use as the architecture's response when capacity pressure is encountered.

Horizontal scaling under human-authored rules. Where the architecture supports it, capacity is expanded horizontally — by adding new substrate instances or partitioning the substrate across instances. The expansion is itself substrate change: a human authors an orchestration rule specifying how new capacity is provisioned and how cells route their reads and writes. The rule is itself substrate-resident authoritative content with its own provenance metadata; later inspection can retrace the expansion through the same path-retraceability mechanisms any other substrate change supports.

Migration to a higher-capacity host under tool-agnosticism. When horizontal scaling within the current host is infeasible or insufficient, migration is admissible. The new host must satisfy the three minimal requirements; substrate content, including all provenance metadata, is moved; orchestration rules and cells are re-pointed. Migration is itself a substrate-level operation under human authorship, recorded with the same provenance metadata that any substrate write carries.

Partitioning and archival under human-authored rules. Where pressure is driven by accumulated content rather than query throughput, partitioning the substrate — typically into an active partition and an archive partition — is admissible. The partitioning rule is human-authored; content moves under the rule, not under vendor or runtime auto-policy. The archive partition is itself substrate; archived content remains addressable; archived provenance remains retrievable. Archival is not deletion. Retraceability is preserved across the partition boundary.

Capacity monitoring and near-limit as governance moment. Capacity indicators are themselves substrate content where the architecture can carry them; reading them is the inspect right exercised over operational metadata. When indicators cross thresholds the deployment has chosen as actionable, the architectural treatment is a governance moment. Orchestration rules may specify alerts, summaries, or other observability outputs. They do not specify automatic load-shedding, automatic pruning, automatic compression, or automatic vendor-managed expansion in the absence of an explicit human-authored rule for that operation. Human authority is exercised in deciding what to do, not in approving an action a runtime has already chosen.

Performance variance as bounded non-determinism; historical-state preservation as non-negotiable. Some performance variance under capacity pressure is admissible without architectural violation: operations may slow within bounded ranges, throughput may dip, recovery may take longer than baseline. This is what the determinism contract admits as bounded non-determinism — performance behaviors that vary within a range the deployment has characterized but do not affect the substrate's state guarantees. The variance is recorded; the bounds are documented. What is not admissible is auto-pruning historical state to relieve pressure. §11.3's substrate-as-source-of-truth commitment, together with retraceability, requires that historical state and provenance be preserved under pressure; archival under a human-authored rule is the mechanism. The architecture does not admit silent erasure of history under any circumstance.

5. Anti-pattern treatments that would violate the architecture

Seven anti-pattern responses violate the architecture at the boundary. Each is named precisely so that downstream remediation can identify which violation is occurring.

Auto-shedding-load-without-recording. The host or runtime drops operations under capacity pressure and the dropped operations are not recorded as substrate state. Both retraceability and the determinism contract are violated. Shedding load may be operationally necessary; doing it silently is the violation. An admissible variant records each shed operation as a substrate event under an explicit rule.

Auto-pruning-historical-state. Historical substrate content is automatically deleted to free capacity. Retraceability and source-of-truth status are violated directly. The architecture admits archival under a human-authored rule; it does not admit auto-deletion. This is the canonical retraceability violation at the capacity boundary.

Capacity-driven-LLM-mediation. An LLM is given authority to decide what substrate state to drop, compress, or summarize under capacity pressure — collapsing older cells into LLM-generated summaries, dropping rationale the LLM judges redundant, reweighting provenance to fit a shrunken

footprint. The substrate-as-source-of-truth commitment fails and the LLM has been instantiated as the source of truth. This is an instance of the broader anti-pattern in which LLM context replaces the substrate as authoritative artifact, surfacing here because capacity pressure makes the substitution operationally tempting.

Vendor-managed-capacity-expansion. The host vendor's auto-scaling features expand capacity without an authoring moment in the substrate's orchestration rules. Governance is violated even when no individual operation visibly fails. The expansion is itself substrate change; vendor auto-expansion bypasses the rule-authoring step the architecture requires for any substrate change.

Silent-degradation. Operations slow or behave inconsistently under capacity pressure but the variance is not recorded as substrate-resident metadata. The determinism contract is violated. Variance within bounded ranges is admissible only if the variance is documented; silent degradation degrades the architecture's observability commitments.

Compression-without-record. Historical substrate content is compressed without preserving the pre-compression form somewhere addressable. Retraceability is violated. The compressed form may be useful as an adjacent representation; it is not admissible as the only form, because compression that is not reversible is functionally indistinguishable from auto-pruning over the lost detail.

Vendor-specific-capacity-features. The response to capacity pressure depends on features of a specific vendor — a scaling tier that exists only in this provider, a non-portable partitioning scheme, a vendor-specific compression format. The substrate is locked to the vendor in a way tool-agnosticism does not admit. The capacity solution must keep the three minimal requirements satisfied across hosts; capacity-driven vendor lock is itself a violation.

6. Performance variance versus architecture violation

A practical operational distinction sits at the heart of the boundary case. Performance variance is not architecture violation. Operations under capacity pressure may slow within documented bounds, may exhibit higher latency variance, may recover from spikes more slowly than at baseline. None of this violates the architecture as long as the variance is bounded, recorded, and within the determinism contract's allowed-variance categories.

What is not admissible is response to capacity pressure that auto-resolves the pressure by bypassing governance, dropping historical state, or locking to a vendor. These responses produce a system faster than it would otherwise be, but at the cost of the architecture's own properties. The line that separates variance from violation runs along three axes: whether operations complete or fail silently; whether historical state is preserved or auto-erased; whether decisions about the response are made by humans authoring rules or by runtime auto-policy.

7. Operational implications

Three implications follow. **Capacity decisions are governance events.** Provisioning new capacity, migrating to a new host, partitioning the substrate, archiving content — each is a substrate-level

operation that requires human authoring of the corresponding orchestration rule. None is a runtime-only or vendor-only event; each surfaces in the substrate's provenance and is inspectable by anyone holding the inspect right within authorized scope. **Horizontal scaling and vendor migration are legitimate.** The architecture admits both responses without distinction in legitimacy. The choice is a deployment decision; tool-agnosticism is what makes vendor migration a normal substrate-level operation rather than a major architectural rebuild. **Archival preserves history while managing capacity.** When storage pressure is the binding constraint, archival under a human-authored rule is the mechanism. The archived partition is substrate; its content is addressable; its provenance is preserved. Capacity is managed; history is not lost. This is the answer the architecture gives to the question that auto-pruning answers wrongly.

8. Limits of the architectural treatment

The boundary case formalized here applies specifically to substrate-level capacity. It does not apply to:

- **LLM consultation capacity** — rate limits, token caps, model availability ceilings imposed by the LLM provider. This is the subject of a separate Phase A6 boundary case; LLM-side pressure stresses the AI-as-substrate-mediator commitment in a way substrate-side pressure does not.
- **Composition partner capacity** — capacity limits in adjacent components a CKS deployment composes with. Also a separate Phase A6 boundary; partner pressure stresses composition-pattern commitments rather than the substrate's own scaling.
- **Vendor-level outage** — host or vendor unavailability that prevents substrate operations from completing at all. A separate Phase A6 boundary; outage is a discontinuity in operation, while near-capacity is a continuous degradation of operating margins.
- **Network partition** — connectivity loss between substrate hosts or between cells and the substrate. The subject of A6.08; partition stresses availability against the determinism contract in a way capacity pressure does not.

Naming the substrate-level capacity boundary as standalone is what allows downstream work to apply the right response to the right boundary; conflating capacity pressure with vendor outage, LLM rate limiting, partner failure, or partition produces architecturally incoherent responses.

9. Operational test

A deployment handles the substrate near-capacity boundary in a CKS-coherent way if and only if all of the following are true at all times during capacity pressure:

- 1 Capacity decisions — expansion, migration, partitioning, archival — are made under human-authored orchestration rules; no vendor or runtime auto-policy makes these decisions on the deployment's behalf without an explicit rule that authorizes the auto-policy.

- 2 No historical substrate content or provenance is auto-deleted to relieve capacity pressure; archival under a human-authored rule is the only mechanism by which content moves out of the active substrate, and archived content remains addressable and retrievable.
- 3 No LLM operation is given authority to decide what substrate state to drop, compress, or summarize in response to capacity pressure.
- 4 The capacity solution does not depend on vendor-specific features that would violate the three minimal requirements of tool-agnosticism; the solution remains portable across hosts that satisfy the requirements.
- 5 Performance variance under capacity pressure is recorded as substrate-resident metadata and is bounded within ranges the deployment has characterized; silent degradation is treated as a defect.

A deployment that fails any of (1)–(5) has not handled the substrate near-capacity boundary in a CKS-coherent way, even if the immediate capacity pressure is relieved. The relief in such a case has been purchased with an architectural property the pattern is not free to spend.

10. Conclusion

The substrate near-capacity boundary is the boundary case in which the temptation to auto-resolve is strongest. Capacity pressure is operationally common at scale; auto-resolution patterns are widely available in surrounding data-platform infrastructure; the immediate pressure relief they offer is real. The architectural commitment the CKS pattern makes — that capacity is handled through human-authored orchestration rules, that historical state is preserved, that performance variance is recorded as bounded non-determinism rather than silently absorbed — is what distinguishes the pattern's response from the auto-resolution defaults available in the host environment.

Naming the boundary as standalone matters because it stresses linear-cost scaling, tool-agnosticism, path retraceability, and human governance jointly. A deployment that approaches capacity is most likely to silently lose architectural integrity precisely at this point, because each of the seven anti-patterns named in §5 is plausible as an immediate-relief mechanism. Standalone formalization gives downstream implementers the precise specification of what is admissible and what is not, and gives downstream remediation a precise vocabulary for naming which violation is occurring when an architectural property is being lost under pressure.

A6.07 follows A6.01–A6.06 as the seventh of approximately fifteen Phase A6 boundary case notes. The next note (A6.08) formalizes the network partition boundary. Subsequent Phase A6 notes will cover further boundary cases. The collected Phase A6 notes together specify what the pattern requires under operational discontinuity at each surface where discontinuity is encountered. Subsequent work that uses different responses to substrate near-capacity is using a different architecture, and the difference should be named.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Substrate Near-Capacity Boundary as Standalone Architectural Treatment in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.